

Structuring Agents, Skills, and MCPs: What We Can Learn from Anthropic's Projects

Introduction

Building a single AI agent that does one thing well is straightforward. Building a system of agents — composable, secure, and maintainable across an organization — is an architectural problem. The challenge is not making the model smarter but designing the scaffolding around it: what knowledge lives where, who can touch what, and how work flows between components without collapsing into a monolith or fragmenting into chaos.

This essay examines a three-layer architecture for agentic systems — skills, agents, and MCP connectors — and distills the design principles that make it work. The ideas and patterns discussed here are derived from Anthropic's reference implementation for finance agents, described in detail at <https://www.anthropic.com/news/finance-agents>. The examples are drawn from that financial-services reference implementation, but the patterns generalize to any domain where agents handle sensitive data, compose domain expertise, and produce work product that humans must ultimately approve.

The Three Layers

The core insight is a clean separation of concerns into three distinct layers, each with a different owner, lifecycle, and trust level:

Skills are units of reusable domain expertise. A skill teaches Claude how to do something — how to build a DCF model, how to trace a GL break, how to draft an IC memo. Skills are passive: they don't decide when to fire or in what order. They are authored once by domain experts, stored in a canonical location, and syndicated to any agent that needs them.

Agents are workflow orchestrators with identity. An agent knows what to do, in what order, and with what constraints. It has a persona ("You are the GL Reconciler"), a defined set of deliverables, a numbered workflow, and guardrails that constrain its behavior. Agents consume skills the way a project manager consumes the expertise of specialists on the team.

MCP connectors are the data layer — standardized integrations that wire agents to external systems (market data terminals, document stores, internal ledgers) without embedding connection logic in the agent or skill itself.

The power of this separation is that each layer evolves independently. A domain expert can improve a skill without touching any agent. An architect can restructure an agent's workflow without rewriting the skills it uses. An IT team can swap a data provider by changing an MCP URL without either the agent or the skills knowing about it.

Skill Design: Author Once, Syndicate Everywhere

Single-source authorship

The most important rule in skill design is that every skill has exactly one canonical source. In practice, this means skills are authored in a domain-organized library — grouped by vertical (investment banking, equity research, fund administration) — and copied into each agent that uses them via an automated sync process.

This matters because the alternative — letting each agent maintain its own version of a skill — guarantees drift. Two agents that both do comps analysis will, within weeks, develop subtly different conventions for how they handle outliers or format output. A sync script that propagates changes from the canonical source into every agent bundle, combined with a linter that fails the build if any copy has drifted, eliminates this class of problem mechanically.

Skills are passive, not agentic

A well-designed skill does not contain workflow logic. It does not say "first do X, then do Y." It says "when you are doing X, here is how to do it well." The skill for comparable company analysis describes metric definitions, outlier treatment, and formatting conventions — it does not decide when in a pitch workflow comps should be run. That decision belongs to the agent.

This passivity is what makes skills composable. The same comps-analysis skill can be invoked at step 4 of a pitch workflow, step 2 of an earnings review, or on demand via a /comps slash command. If the skill contained sequencing logic, it would be coupled to a single workflow and unusable elsewhere.

Slash commands as skill entry points

Skills can optionally expose slash commands (/comps, /dcf, /earnings) that let users invoke them directly. This is the "toolkit" mode — install the vertical plugin, get the commands, use them ad hoc without any agent orchestration. It's a useful on-ramp: users who aren't ready for full agent workflows can still benefit from structured domain expertise.

Agent Design: Identity, Workflow, Guardrails

The agent prompt template

A well-structured agent prompt has five sections, always in the same order:

Frontmatter — machine-readable metadata: name, description, tool allowlist
Identity — who the agent is and what it owns

Deliverables — the exact artifacts the agent produces (not vague goals — concrete outputs)

Workflow — numbered steps, each referencing specific skills by name

Guardrails — hard constraints on what the agent must not do

This consistency matters for maintainability. When every agent follows the same template, a new team member can read any agent prompt and immediately understand its scope, its process, and its limits. It also makes it possible to lint and validate agent prompts programmatically.

Explicit deliverables over vague mandates

The difference between a useful agent and a dangerous one often comes down to how precisely its outputs are defined. Compare "help with GL reconciliation" to:

Given a trade date and list of asset classes, you deliver: **Break list** — every GL/subledger variance over threshold, with account, balances, variance, suspected cause.

Root-cause trace — for each break, the transaction-level evidence and classification.

Exception report — formatted for controller sign-off, with recommended resolution per break.

The first framing invites scope creep and hallucination. The second gives the agent — and the human reviewing its output — a concrete checklist to evaluate against. When deliverables are explicit, the agent knows when it's done, and the reviewer knows what to check.

Workflow as skill composition

The workflow section is where agents earn their keep. It transforms a bag of skills into an ordered process with dependencies, handoffs, and quality gates. Each step typically maps to one skill invocation:

1. **Scope the ask.** Confirm target, sector, and situation.
2. **Write the situation overview.** Invoke the sector-overview skill.
3. **Pull data.** Use the CapIQ MCP for trading multiples and filings.
4. **Spread the peer set.** Invoke the comps-analysis skill.
5. **Build the model.** Invoke dcf-model and 3-statement-model.

Notice that the workflow references skills and data sources by name. This makes the dependencies explicit and auditable. It also means the workflow can be understood without reading the skill files — the agent prompt is a self-contained specification of the process.

Guardrails: what the agent must never do

Every agent should declare its negative space — the things it will not do under any circumstances. These fall into recurring categories:

No system-of-record writes. An agent that reconciles a ledger produces a report; it does not post journal entries. An agent that screens KYC documents recommends a risk rating; it does not approve onboarding. The line between "draft" and "execute" is the most important guardrail in any financial workflow.

No external communications. Agents that produce client-facing work (pitch decks, research notes) have no email or messaging tools. Distribution is a human decision.

Source everything. When a number cannot be traced to a data source, the agent flags it as [UNSOURCED] rather than estimating. This turns hallucination from a silent failure into a visible one.

Human approval gates. The agent stops and surfaces work for review at defined checkpoints — after the model is built, after the deck is generated — rather than running to completion unsupervised. This is especially important for multi-artifact workflows where an error in an early stage compounds downstream.

Security Through Tiered Isolation

The untrusted-content problem

In many real workflows, the agent must process documents authored by outsiders — counterparty statements, onboarding packets, custodian reports. These documents may contain adversarial instructions designed to manipulate the agent (prompt injection). The architectural question is: how do you let the agent read untrusted content without letting that content reach your tools or systems?

Subagent tiers

The answer is to split the agent into tiered subagents with different trust levels and tool access. The pattern has three tiers:

Reader tier — touches untrusted documents. Has Read and Grep only. No MCP access, no write tools, no bash. Its output is schema-validated with length caps and character-class restrictions, so injected instructions cannot survive intact into downstream tiers. This is the quarantine zone.

Orchestrator tier — never touches untrusted documents directly. Dispatches readers, aggregates their (validated) output, and coordinates the workflow. Has read tools plus MCP access to trusted internal systems. Cannot write.

Resolver tier — the only tier with write access. Receives verified, critic-checked data and produces the final output artifacts. Never opens outsider files, never runs bash, never connects to external systems. Its attack surface is

minimal because it only sees data that has already been validated by two upstream tiers.

A fourth role, the critic, adds an independent verification step: it re-checks each finding against trusted sources before the resolver writes anything. This catch layer is particularly valuable in reconciliation and audit workflows where false positives are costly.

Deny-by-default tooling

The mechanical enforcement of this tiering is deny-by-default tool configuration. Every subagent starts with all tools disabled and explicitly enables only the ones it needs:

yaml

tools:

```
- type: agent_toolset_20260401
  default_config:
    enabled: false    # everything off
  configs:
    - name: read
      enabled: true    # explicitly granted
    - name: grep
      enabled: true    # explicitly granted
```

This is the principle of least privilege applied to agentic systems. An agent that doesn't need write access doesn't get write access — not as a prompt instruction that can be overridden, but as a mechanical config that the model cannot change at runtime.

MCP Connectors: Centralized Data Access

One source, many consumers

MCP (Model Context Protocol) connectors should be defined once in a shared location and consumed by any agent or skill that needs them. In practice, this means a single `.mcp.json` file in a core plugin that lists every data provider, and all other plugins inherit from it.

This centralization prevents the proliferation of connection configs across agents. When a data provider changes its URL or authentication scheme, you update one file, not ten. It also makes it easy to audit what external systems your agents can reach — the answer is always in one place.

Connector access as a security boundary

MCP access is a meaningful security boundary. In the tiered subagent model, connector access is granted selectively:

- The **reader** (which touches untrusted content) gets no MCP access — it cannot reach any firm system
- The **orchestrator** gets read-only MCP access to trusted internal systems.
- The **resolver** (which can write files) gets no MCP access — it cannot exfiltrate data.

This creates a separation where the agent that can read external data cannot write to the filesystem, and the agent that can write to the filesystem cannot read external data. A compromised reader cannot use MCP to reach firm systems; a compromised resolver cannot use MCP to send data anywhere.

Composition Patterns

Agent-to-agent handoffs

Complex workflows often span multiple agents. A GL reconciliation may discover breaks that need to be addressed by a month-end close process. Rather than expanding the reconciler's scope, the agent emits a structured `handoff_request` that an orchestration layer routes to the appropriate downstream agent.

This handoff pattern preserves each agent's single responsibility while enabling multi-agent pipelines. The key constraint is that handoffs must go through a validated allowlist — an agent cannot arbitrarily invoke any other agent. The orchestration layer (whether a Python script, Temporal workflow, or Airflow DAG) validates the handoff target and payload before routing.

Dual deployment from one source

A well-structured agent should deploy identically to interactive environments (where a human is in the loop) and headless environments (where the agent runs autonomously via API). This is achieved by keeping the agent's core definition — system prompt and skills — in the plugin directory, and adding a thin deployment wrapper (an `agent.yaml` with environment-specific config) for the headless path.

The headless wrapper adds things like MCP server URLs (which are environment-specific), subagent manifests (which are deployment-specific), and a one-line append to the system prompt ("You are running headless. Produce files in `./out/`; do not assume an open Office document."). The core identity, workflow, and guardrails remain identical.

Summary of Principles

1. Separate knowledge from orchestration. Skills teach how; agents decide what, when, and with what constraints.
2. Author skills once, syndicate mechanically. A sync script and drift linter prevent divergence.
3. Make skills passive. No sequencing logic in skills — that belongs to the agent.
4. Define deliverables, not goals. Concrete output specs prevent scope creep and enable review.
5. Declare negative space. Every agent should state what it will never do.
6. Deny tools by default. Grant the minimum toolset each tier needs, mechanically.
7. Isolate untrusted content structurally. The reader that opens outsider documents cannot reach your tools or systems.
8. Validate at boundaries. Schema-validate and length-cap every output that crosses a trust tier.
9. Centralize data access. One MCP config, consumed by many — auditable and swappable
10. Deploy from one source. The same prompt and skills should power both interactive and headless modes. Deploy from one source.

These are not theoretical principles — they are patterns extracted from Anthropic's [finance agents reference implementation](#), a working system that handles financial data, processes untrusted documents, and produces work product that humans sign off on. The common thread is that good agent architecture is less about making the model more capable and more about making the system around it more predictable, auditable, and safe.



Structuring Agents, Skills, and MCPs: Best Practices from the Field

INTRODUCTION

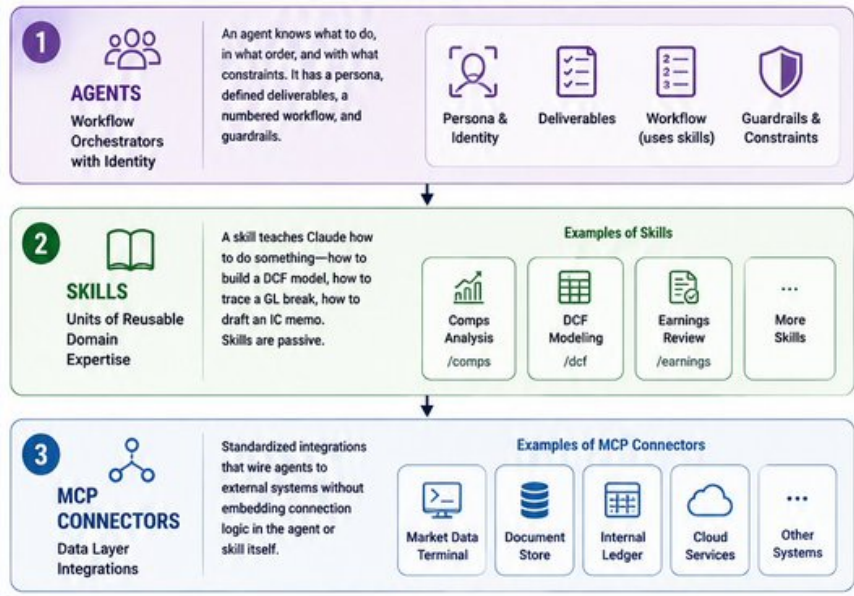
Building a single AI agent that does one thing well is straightforward. Building a system of agents—composable, secure, and maintainable across an organization—is an architectural problem. The challenge is not making the model smarter but designing the scaffolding around it: what knowledge lives where, who can touch what, and how work flows between components without collapsing into a monolith or fragmenting into chaos.



The core insight:

A clean separation of concerns into three distinct layers, each with a different owner, lifecycle, and trust level.

THE THREE-LAYER ARCHITECTURE



Independent Evolution:

Each layer evolves independently.



Improve a skill without touching any agent.



Restructure an agent's workflow without rewriting skills.



Swap a data provider by changing an MCP URL—no changes to agents or skills.

SKILL DESIGN: AUTHOR ONCE, SYNDICATE EVERYWHERE



1. SINGLE-SOURCE AUTHORSHIP

Every skill has exactly one canonical source in a domain-organized library (by vertical). Skills are copied into each agent via automated sync.

- Prevents drift and inconsistent conventions
- Sync script propagates changes
- Linter fails the build if any copy has drifted



2. SKILLS ARE PASSIVE, NOT AGENTIC

A skill does not contain workflow logic. It says "when you are doing X, here is how to do it well."

- No sequencing ("first X, then Y")
- Describes best practices, definitions, formats, conventions
- Composable across any workflow



3. SLASH COMMANDS AS ENTRY POINTS

Skills can expose slash commands (/comps, /dcf, /earnings) for ad hoc use—"toolkit mode."

- Install the vertical plugin, get the commands
- Use structured domain expertise on demand
- On-ramp to full agent workflows

AGENT DESIGN: IDENTITY, WORKFLOW, GUARDRAILS



THE AGENT PROMPT TEMPLATE

(Always in this order)

- 1 Frontmatter – machine-readable metadata: name, description, tool allowlist
- 2 Identity – who the agent is and what it owns
- 3 Deliverables – the exact artifacts the agent produces
- 4 Workflow – numbered steps, each referencing specific skills by name
- 5 Guardrails – hard constraints on what the agent must not do



EXPLICIT DELIVERABLES OVER VAGUE MANDATES



Vague (dangerous)

"Help with GL reconciliation."

→ Invites scope creep and hallucination.



Explicit (effective)

Given a trade date and list of asset classes, you deliver:

1. Break list – every GL/subledger variance over threshold, with account, balances, variance, suspected cause.
2. Root-cause trace – for each break, the transaction-level evidence and classification.
3. Exception report – formatted for controller sign-off, with recommended resolution per break.

KEY PRINCIPLES THAT MAKE IT WORK



Separation of Concerns
Each layer has a clear purpose and evolves independently.



Single Source of Truth
Author once, syndicate everywhere, eliminate drift.



Composability
Passive skills + workflow agents = reusable, flexible systems.



Security by Design
Guardrails, tool allowlists, and MCP boundaries.



Measurable Outputs
Explicit deliverables create accountability and quality.



Human in the Loop
Agents produce, humans approve.



The result: Agentic systems that are composable, secure, and maintainable at enterprise scale—where domain experts own the knowledge, architects own the workflows, and IT owns the connections.

Source: Structuring Agents, Skills, and MCPs: Best Practices from the Field ([filecite:turn0file03](#))

